

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	09 - 15 - 26	DATE SUBMITTED:	09 - 20 - 26
NAME:	Santiago, David Owen A.	INSTRUCTOR:	Engr. Jimlord Quejado

Activity 8.1	Install, Configure, and Manage Availability Monitoring tools
1. PROCEDURE OUTPUT	
Step 1: Create a playbook that installs Nagios in both Ubuntu and CentOS. Apply the concept of creating roles.	
Step 2: Describe how you did step 1. (<i>Provide screenshots and explanations in your report. Make your report detailed such that it will look like a manual.</i>)	
Nagios Installation for Ubuntu and CentOS (install_nagios.yml): Pretasks: This ensures the target nodes are available and their software is up to date.	
<pre> - hosts: all become: true pre_tasks: - name: install updates (CentOS) tags: always dnf: update_only: yes update_cache: yes when: ansible_distribution == "CentOS" - name: install updates (Ubuntu) tags: always apt: upgrade: dist </pre>	

```
update_cache: yes
when: ansible_distribution == "Ubuntu"
```

Handlers:

Defines the systemd restart routines for the Web Server (Apache) and Nagios, which can be called with `notify`:

```
handlers:
  - name: Restart Web Server
    ansible.builtin.systemd:
      name: "{{ web_server_service }}"
      state: restarted

  - name: Restart Nagios
    ansible.builtin.systemd:
      name: nagios
      state: restarted
```

0. **Step 0 - Configure SELinux:** Sets SELinux to permissive mode strictly for RedHat-based systems to prevent access control blocks during installation.

```
- name: Step 0 - Set SELinux to permissive mode on RedHat systems
  ansible.posix.selinux:
    policy: targeted
    state: permissive
  when: ansible_facts['os_family'] == 'RedHat'
```

1. **Step 1 - Install prerequisite packages:** Installs required compilers, libraries, PHP, and web server packages using `apt` for Debian/Ubuntu and `dnf` for CentOS/RHEL. On RedHat systems, it also enables the EPEL and CRB/PowerTools repositories to install the `perl-Net-SNMP` package.

```
- name: Step 1 - Install Nagios Core prerequisite packages
  (Ubuntu/Debian)
  ansible.builtin.apt:
```

```
name:
  - autoconf
  - gcc
  - libc6
  - make
  - wget
  - unzip
  - apache2
  - php
  - libapache2-mod-php
  - libgd-dev
  - ufw
  - openssl
  - libssl-dev
state: present
update_cache: yes
when: ansible_facts['os_family'] == 'Debian'

# Step 1: Install prerequisite packages for CentOS/RHEL
- name: Step 1 - Install Nagios Core prerequisite packages
  (CentOS/RHEL)
  ansible.builtin.dnf:
    name:
      - gcc
      - glibc
      - glibc-common
      - make
      - gettext
      - automake
      - autoconf
      - gd
      - gd-devel
      - perl
      - net-snmp
      - net-snmp-utils
```

```
- openssl-devel
- epel-release
- wget
- tar
- httpd
- php
- php-cli
- firewallld
state: present
when: ansible_facts['os_family'] == 'RedHat'

- name: Step 1 - Install perl-Net-SNMP with PowerTools/CRB and EPEL
  enabled (CentOS/RHEL)
  ansible.builtin.dnf:
    name: perl-Net-SNMP
    enablerepo:
      - epel
      - "{{ 'crb' if ansible_facts['distribution_major_version'] |
int >= 9 else 'powertools' }}"
    state: present
    when: ansible_facts['os_family'] == 'RedHat'
```

2. **Step 2 - Download and extract Nagios Core:** Creates a `/tmp/nagios-build` directory and downloads the Nagios Core 4.4.14 source archive from GitHub, automatically extracting the contents into the build directory.

```
- name: Step 2 - Create build directory for Nagios Core
  ansible.builtin.file:
    path: /tmp/nagios-build
    state: directory
    mode: '0755'

- name: Step 2 - Download and extract Nagios Core source archive
  ansible.builtin.unarchive:
```

```
src:
https://github.com/NagiosEnterprises/nagioscore/releases/download/nagios-4.4.14/nagios-4.4.14.tar.gz
dest: /tmp/nagios-build
remote_src: yes
extra_opts: [--strip-components=1]
creates: /tmp/nagios-build/configure
```

3. **Step 3 - Configure and compile Nagios Core:** Executes the `./configure` script to prepare the build environment (adding specific Apache flags for Debian systems) and compiles the core binaries using `make all`.

```
- name: Step 3 - Run configure script (Ubuntu/Debian)
  ansible.builtin.command:
    cmd: ./configure --with-httpd-conf=/etc/apache2/sites-enabled
    chdir: /tmp/nagios-build
    creates: /tmp/nagios-build/Makefile
    when: ansible_facts['os_family'] == 'Debian'

- name: Step 3 - Run configure script (CentOS/RHEL)
  ansible.builtin.command:
    cmd: ./configure
    chdir: /tmp/nagios-build
    creates: /tmp/nagios-build/Makefile
    when: ansible_facts['os_family'] == 'RedHat'

- name: Step 3 - Compile Nagios Core
  ansible.builtin.command:
    cmd: make all
    chdir: /tmp/nagios-build
    creates: /tmp/nagios-build/base/nagios
```

4. **Step 4 - Create users and groups:** Uses `make install-groups-users` to generate the system accounts necessary for Nagios, and appends the

OS-specific web server user (`www-data` or `apache`) to the new `nagios` group.

```
- name: Step 4 - Install Nagios user and group
  ansible.builtin.command:
    cmd: make install-groups-users
    chdir: /tmp/nagios-build
  register: nagios_user_check
  changed_when: false

- name: Step 4 - Add web server user to nagios group
  ansible.builtin.user:
    name: "{{ web_server_user }}"
    groups: nagios
    append: yes
```

5. **Step 5 - Install binary files:** Deploys the compiled Nagios core program files into their designated system paths via `make install`.

```
- name: Step 5 - Install binaries
  ansible.builtin.command:
    cmd: make install
    chdir: /tmp/nagios-build
  creates: /usr/local/nagios/bin/nagios
```

6. **Step 6 - Install daemon / systemd service:** Generates and installs the systemd service file (`nagios.service`) required to run Nagios as a background process via `make install-daemoninit`.

```
- name: Step 6 - Install systemd service
  ansible.builtin.command:
    cmd: make install-daemoninit
    chdir: /tmp/nagios-build
  creates: /lib/systemd/system/nagios.service
```

7. **Step 7 - Install command mode permissions:** Configures the necessary directory permissions that allow the web interface to submit external commands to the core monitoring engine via `make install-commandmode`.

```
- name: Step 7 - Install external command mode permissions
  ansible.builtin.command:
    cmd: make install-commandmode
    chdir: /tmp/nagios-build
    creates: /usr/local/nagios/var/rw
```

8. **Step 8 - Install sample configuration files:** Installs the default template configurations (like `nagios.cfg`) required for the engine to initialize properly via `make install-config`.

```
- name: Step 8 - Install sample configuration files
  ansible.builtin.command:
    cmd: make install-config
    chdir: /tmp/nagios-build
    creates: /usr/local/nagios/etc/nagios.cfg
```

9. **Step 9 - Install Apache web config and enable modules:** Places the Apache configuration file for the Nagios interface into the appropriate directory based on the OS, and explicitly enables the `rewrite` and `cgi` modules for Debian systems.

```
- name: Step 9 - Install Apache web configuration
  ansible.builtin.command:
    cmd: make install-webconf
    chdir: /tmp/nagios-build
    creates: "{{ '{ ' /etc/apache2/sites-enabled/nagios.conf' if
ansible_facts['os_family'] == 'Debian' else
'/etc/httpd/conf.d/nagios.conf' }}"
  notify: Restart Web Server
```

```
- name: Step 9 - Enable Apache rewrite and cgi modules
(Ubuntu/Debian only)
  community.general.apache2_module:
    name: "{{ item }}"
    state: present
  loop:
    - rewrite
    - cgi
  when: ansible_facts['os_family'] == 'Debian'
  notify: Restart Web Server
```

10. Step 10 - Configure Firewall: Modifies the local firewall to permit inbound HTTP and HTTPS traffic, utilizing UFW for Debian systems and Firewallld for RedHat systems.

```
- name: Step 10 - Allow Apache traffic through UFW (Ubuntu/Debian)
  community.general.ufw:
    rule: allow
    name: Apache
  when: ansible_facts['os_family'] == 'Debian'

- name: Step 10 - Reload UFW rules (Ubuntu/Debian)
  community.general.ufw:
    state: reloaded
  when: ansible_facts['os_family'] == 'Debian'

- name: Step 10 - Allow HTTP and HTTPS services through Firewallld
(CentOS/RHEL)
  ansible.posix.firewalld:
    service: "{{ item }}"
    permanent: yes
    state: enabled
    immediate: yes
```

```
loop:
  - http
  - https
when: ansible_facts['os_family'] == 'RedHat'
```

11. **Step 11 - Create web login user:** Executes a shell command to generate an `htpasswd` file, securely injecting the `nagios_user` and `nagiosadmin_password` variables to create the web dashboard authentication credentials.

```
- name: Step 11 - Create htpasswd file for nagiosadmin
  ansible.builtin.shell:
    cmd: htpasswd -b -c /usr/local/nagios/etc/htpasswd.users '{{
nagios_user }}' '{{ nagiosadmin_password }}'
    creates: /usr/local/nagios/etc/htpasswd.users
```

12. **Step 12 - Ensure Web Server service is started:** Uses the `systemd` module to enable and start the Apache web server (`apache2` or `httpd`).

```
- name: Step 12 - Ensure Web Server service is running
  ansible.builtin.systemd:
    name: "{{ web_server_service }}"
    state: started
    enabled: yes
```

13. **Step 13 - Start Nagios service:** Uses the `systemd` module to enable and start the newly configured Nagios monitoring engine.

```
- name: Step 13 - Enable and start Nagios service
  ansible.builtin.systemd:
    name: nagios
    state: started
    enabled: yes
```

14. **Step 14 - Install Nagios Plugins prerequisites:** Installs additional compilation dependencies specifically required for building the Nagios Plugins on Debian/Ubuntu systems.

```
- name: Step 14 - Install Nagios Plugins prerequisite packages
(Ubuntu/Debian)
  ansible.builtin.apt:
    name:
      - autoconf
      - gcc
      - libc6
      - libmccrypt-dev
      - make
      - libssl-dev
      - wget
      - bc
      - gawk
      - dc
      - build-essential
      - snmp
      - libnet-snmp-perl
      - gettext
      - iputils-ping
    state: present
    when: ansible_facts['os_family'] == 'Debian'
```

15. **Step 15 - Download and extract Nagios Plugins:** Creates a `/tmp/nagios-plugins-build` directory and downloads the Nagios Plugins 2.4.8 source archive, extracting it directly into the directory.

```
- name: Step 15 - Create build directory for Nagios Plugins
  ansible.builtin.file:
    path: /tmp/nagios-plugins-build
    state: directory
    mode: '0755'
```

```
- name: Step 15 - Download and extract Nagios Plugins archive
ansible.builtin.unarchive:
  src:
https://github.com/nagios-plugins/nagios-plugins/releases/download/release-2.4.8/nagios-plugins-2.4.8.tar.gz
  dest: /tmp/nagios-plugins-build
  remote_src: yes
  extra_opts: [--strip-components=1]
  creates: /tmp/nagios-plugins-build/configure
```

16. Step 16 - Configure, compile, and install Nagios Plugins: Prepares the build environment via `./configure`, compiles the plugin source code, and installs the binaries into the Nagios `libexec` directory. Upon successful completion, it triggers the restart handlers for both Nagios and the Web Server to apply all final changes.

```
- name: Step 16 - Configure Nagios Plugins
ansible.builtin.command:
  cmd: ./configure
  chdir: /tmp/nagios-plugins-build
  creates: /tmp/nagios-plugins-build/Makefile

- name: Step 16 - Compile Nagios Plugins
ansible.builtin.command:
  cmd: make
  chdir: /tmp/nagios-plugins-build
  creates: /tmp/nagios-plugins-build/plugins/check_ping

- name: Step 16 - Install Nagios Plugins
ansible.builtin.command:
  cmd: make install
  chdir: /tmp/nagios-plugins-build
  creates: /usr/local/nagios/libexec/check_ping
notify:
```

- Restart Nagios
- Restart Web Server

GITHUB Links:

Install_Nagios.yml Repository:

<https://github.com/owen-s-dotdev/CPE-Santiago-Ansible-Projects.git>

CPE212-Santiago Repository:

<https://github.com/owen-s-dotdev/CPE212-Santiago.git>

Step 3: Show an output of the installed Nagios for both Ubuntu and CentOS.

```
TASK [Step 17 - Install Nagios Plugins] *****
changed: [ubuntu1]
changed: [centos1]

RUNNING HANDLER [Restart Web Server] *****
changed: [ubuntu1]
changed: [centos1]

RUNNING HANDLER [Restart Nagios] *****
changed: [ubuntu1]
changed: [centos1]

PLAY RECAP *****
centos1      : ok=26   changed=22   unreachable=0    failed=0    skipped=6    rescued=0
ignored=0
ubuntu1      : ok=26   changed=19   unreachable=0    failed=0    skipped=6    rescued=0
ignored=0

santiago@workstation:~/CPE212-Santiago/ansible-lab$
```

VERIFICATION

Ubuntu:

Ubuntu26server1 (09-20-26 (Before NAGIOS)) [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Sep 20 17:54



Nagios: 127.0.0.1

+

← → ↻

http://127.0.0.1/nagios/

☆



Nagios®

General

Home

Documentation

Current Status

Tactical Overview

Map (Legacy)

Hosts

Services

Host Groups

Summary

Grid

Service Groups

Summary

Grid

Problems

Services (Unhandled)

Hosts (Unhandled)

Network Outages

Quick Search:

Reports

Availability

Trends (Legacy)

Alerts

History

Summary

Histogram (Legacy)

Notifications

Event Log

System

Comments

Downtime

Nagios® Core™

✓ Daemon running with PID 30052

Nagios® Core™

Version 4.4.14

August 01, 2023

Check for updates

A new version of Nagios Core is available!

Visit nagios.org to download Nagios 4.5.14.

Get Started

- Start monitoring your infrastructure
- Change the look and feel of Nagios
- Extend Nagios with hundreds of addons
- Get support
- Get training
- Get certified

Latest News

Quick Links

- Nagios Library (tutorials and docs)
- Nagios Labs (development blog)
- Nagios Exchange (plugins and addons)
- Nagios Support (tech support)
- Nagios.com (company)
- Nagios.org (project)

Don't Miss...

Page Tour

CentOS9 (09-20-26 (Before NAGIOS)) [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Sep 20 05:53



Nagios: 127.0.0.1

+

← → ↻

http://127.0.0.1/nagios/

☆



Nagios®

General

Home

Documentation

Current Status

Tactical Overview

Map (Legacy)

Hosts

Services

Host Groups

Summary

Grid

Service Groups

Summary

Grid

Problems

Services (Unhandled)

Hosts (Unhandled)

Network Outages

Quick Search:

Reports

Availability

Trends (Legacy)

Alerts

History

Summary

Histogram (Legacy)

Notifications

Event Log

System

Comments

Downtime

Nagios® Core™

✓ Daemon running with PID 75941

Nagios® Core™

Version 4.4.14

August 01, 2023

Check for updates

A new version of Nagios Core is available!

Visit nagios.org to download Nagios 4.5.14.

Get Started

- Start monitoring your infrastructure
- Change the look and feel of Nagios
- Extend Nagios with hundreds of addons
- Get support
- Get training
- Get certified

Latest News

Quick Links

- Nagios Library (tutorials and docs)
- Nagios Labs (development blog)
- Nagios Exchange (plugins and addons)
- Nagios Support (tech support)
- Nagios.com (company)
- Nagios.org (project)

Don't Miss...

Tour

Make sure to create a new repository in GitHub for this activity.

Links:

Repository for `install_nagios.yml`:

<https://github.com/owen-s-dotdev/CPE-Santiago-Ansible-Projects.git>

CPE212-Santiago Repository:

<https://github.com/owen-s-dotdev/CPE212-Santiago.git>

Ubuntu Installation Steps retrieved from:

[Nagios Ubuntu](#)

CentOS Installation Steps retrieved from:

[Nagios CentOS](#)

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Answer the following:

1. What are the benefits of having an availability monitoring tool?

Monitoring tools allow us to supervise and track the status and availability in the managed nodes. They provide a centralized interface containing the important information about our managed nodes like hardware information, network ports information, and of course, their statuses. With the live status feed, system administrators can easily detect and fix any sudden errors to minimize system downtimes and maintain system operations.

4. CONCLUSION/ANALYSIS/LEARNINGS

In conclusion, this activity introduces one of the basic monitoring tools, Nagios. I learned how to search and create a centralized installation procedure based on the retrieved installation steps from the official Nagios installation documentation. I integrated the installation steps for Ubuntu and CentOS into a single playbook, `install_nagios.yml`, which contains each installation step with the appropriate commands specific to each distribution. Doing this activity made me discover other useful ansible builtin commands like `ansible.builtin.command`, `ansible.builtin.shell`, `ansible.posix.selinux`, `ansible.posix.unarchive`, `ansible.posix.user`, and `community.general` commands like `community.general.apache2_module`, and `community.general.ufw` for more specific commands. Overall, this goes to show how versatile and useful Ansible is as a configuration and automation tool due to the wide range of customization and commands that it can automate.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini AI

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Compilation and structuring of commands retrieved from the researched documentation pages. Debugging of CentOS installation Issues

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Gemini AI was used to assist in code cleaning and debugging. The code structure and steps retrieved were converted to ansible commands on my own.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.